

UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA

INFORME TÉCNICO E INTEGRADOR

**Food Store - Sistema de Gestión de Pedidos de
Comida**

Materia: Programación 2

Docente: Cinthia Rigononi

Estudiante: Matías Límina | Nicolas Monjelardi | Lautaro Aguero

Fecha de Entrega: 22 / 06 / 2026

ÍNDICE

1. Marco Teórico	3
2. Decisiones Técnicas y Arquitectura	6
3. Dificultades y Soluciones	9
4. Demostración del Sistema	12
5. Bibliografía y Webgrafía	15

1. Marco Teórico	
○ 1.1 Java 21 y el Paradigma Orientado a Objetos	
○ 1.2 Bases de Datos Relacionales y MySQL	
○ 1.3 JDBC y el Pool de Conexiones HikariCP	
○ 1.4 Principios de POO Aplicados	
○ 1.5 Patrones de Diseño y Arquitectura de Software	
2. Decisiones Técnicas y Arquitectura	
○ 2.1 Estructura de Paquetes (Modularización)	
○ 2.2 Diseño Físico de la Base de Datos (DDL)	
○ 2.3 Mecanismo de Baja Lógica (Soft Delete)	
○ 2.4 Mapeo de Datos SQL a Objetos Java	
○ 2.5 Arquitectura de Excepciones Personalizadas	
○ 2.6 Robustez del Menú en Consola	
3. Dificultades y Soluciones	
○ 3.1 Transición Metodológica de Colecciones a JDBC Real	
○ 3.2 Bug del Scanner: Excepciones de Entrada Vacía en Menús Numéricos	
○ 3.3 Integridad Referencial y Manejo de Stock Transaccional	
4. Demostración del Sistema (Capturas de Pantalla)	
5. Bibliografía y Webgrafía	

1. MARCO TEÓRICO

El desarrollo del sistema **Food Store** se fundamenta en la integración de conceptos avanzados de programación estructurada, orientada a objetos y persistencia relacional utilizando la plataforma de desarrollo Java.

1.1 Java 21 y el Paradigma Orientado a Objetos

Java 21 es una versión de soporte a largo plazo (LTS) que introduce mejoras sustanciales en rendimiento, concurrencia y legibilidad de sintaxis. El paradigma de la **Programación Orientada a Objetos (POO)** permite modelar el negocio de comidas mediante entidades del mundo real estructuradas como clases. A través de este paradigma, se estructuran tipos de datos complejos que agrupan estado (atributos) y comportamiento (métodos).

1.2 Bases de Datos Relacionales y MySQL

Para garantizar la durabilidad y consistencia de los datos, se prescindió del almacenamiento volátil en memoria y se incorporó un motor de **Base de Datos Relacional (RDBMS)**, específicamente **MySQL**. Este enfoque permite:

- **Persistencia física:** Los datos sobreviven al ciclo de vida de la ejecución del programa.
- **Integridad referencial:** Uso de restricciones de clave primaria (PRIMARY KEY) y claves foráneas (FOREIGN KEY) para modelar asociaciones del mundo real, impidiendo inconsistencias de datos (por ejemplo, registrar un pedido con un usuario inexistente).

1.3 JDBC y el Pool de Conexiones HikariCP

JDBC (Java Database Connectivity) es la API estándar de Java que provee conectividad independiente con bases de datos. Sin embargo, abrir y cerrar conexiones físicas directamente para cada consulta al motor genera un cuello de botella crítico en términos de rendimiento e hilos de procesamiento.

Para mitigar este impacto, se implementó **HikariCP**, un pool de conexiones JDBC de alto rendimiento, extremadamente ligero y rápido. Su principio es mantener un conjunto de conexiones físicas activas ("vivas") en memoria (configurado en nuestro proyecto con un

tamaño máximo de ¹⁰ conexiones). Cuando la aplicación requiere interactuar con MySQL:

1. Toma una conexión disponible del pool en un tiempo de respuesta medido en microsegundos.
2. Ejecuta la sentencia SQL.
3. Devuelve la conexión al pool de manera automática, reduciendo el sobrecosto de latencia asociado al protocolo TCP/IP de inicialización del motor de bases de datos.

1.4 Principios de POO Aplicados

El diseño de software del sistema aplica rigurosamente los pilares de la POO:

- **Encapsulamiento:** Ocultamiento de los detalles de implementación interna de los atributos de las entidades (ej. Usuario, Producto), exponiendo el acceso exclusivamente a través de métodos de interfaz pública (getters y setters) con validaciones de negocio.
- **Herencia:** Creación de una clase base e integradora llamada Base que actúa como superclase. Esta centraliza atributos de auditoría y persistencia (id, eliminado, createdAt), permitiendo que Usuario, Producto, Categoría, Pedido y DetallePedido reutilicen el código base, promoviendo el principio DRY (*Don't Repeat Yourself*).
- **Polimorfismo:** Sobrescritura de métodos del ciclo de vida del objeto raíz como toString(), equals() y hashCode(). El método equals() fue redefinido para comparar la identidad del registro basándose en la correspondencia inequívoca del atributo id de la base de datos relacional.
- **Interfaces:** Definición del contrato conceptual Calculable para estandarizar comportamientos de cálculo de totalización financiera en el módulo de facturación y pedidos.

1.5 Patrones de Diseño y Arquitectura de Software

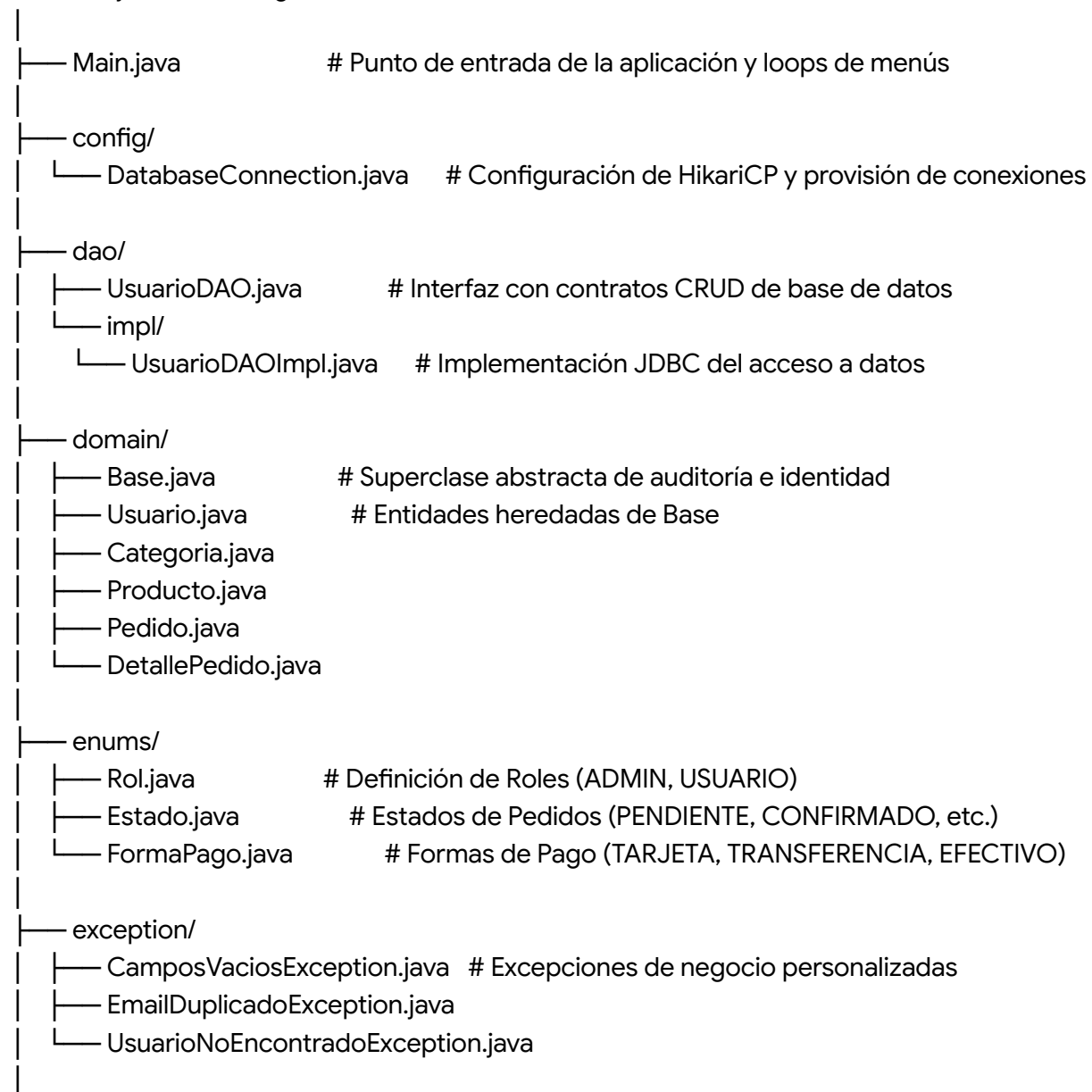
- **Arquitectura en Capas:** Desacoplamiento físico del código fuente para aislar responsabilidades. El sistema se divide en:
 - **Capa de Presentación / UI (Main):** Único responsable de interactuar con el operador a través del flujo de entrada/salida de consola.
 - **Capa de Servicio / Negocio (service):** Orquesta los flujos de control de la aplicación, ejecutando reglas lógicas complejas (ej. validación de unicidad de emails, existencia de stock).
 - **Capa de Persistencia / Acceso a Datos (dao):** Abstrae las llamadas directas SQL al motor transaccional.
 - **Capa de Dominio / Modelo (domain):** Representa el modelo relacional mapeado a clases de Java.
- **Patrón DAO (Data Access Object):** Estructura que separa la lógica de acceso a datos de los servicios de negocio mediante interfaces (UsuarioDAO) e implementaciones concretas (UsuarioDAOImpl), facilitando la mantenibilidad y un eventual cambio de base de datos sin alterar la lógica de negocio.
- **Patrón Singleton:** Utilizado para garantizar que exista una única instancia global del pool de conexiones HikariDataSource en toda la aplicación, concentrado en la clase DatabaseConnection.

2. DECISIONES TÉCNICAS Y ARQUITECTURA

2.1 Estructura de Paquetes (Modularización)

El código se estructuró de manera estandarizada bajo el gestor de dependencias **Maven**, organizando los componentes del paquete base `com.integrador.foodstore` en las siguientes rutas físicas:

`src/main/java/com/integrador/foodstore/`



```
└─ service/
  └─ UsuarioService.java      # Capa lógica de orquestación y validaciones
```

2.2 Diseño Físico de la Base de Datos (DDL)

La base de datos física del sistema se aloja en un esquema relacional con restricciones estrictas de claves foráneas. El archivo de migración SQL se encuentra estructurado en la raíz de los recursos del proyecto (src/main/resources/database.sql):

```
CREATE DATABASE IF NOT EXISTS foodstore_db;
USE foodstore_db;
```

```
CREATE TABLE IF NOT EXISTS usuarios (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  eliminado TINYINT(1) NOT NULL DEFAULT 0,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  nombre VARCHAR(100) NOT NULL,
  apellido VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  celular VARCHAR(10) NOT NULL,
  password VARCHAR(255) NOT NULL,
  rol VARCHAR(30) NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS categorias (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  eliminado TINYINT(1) NOT NULL DEFAULT 0,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  nombre VARCHAR(100) NOT NULL,
  descripcion VARCHAR(255)
);
```

```
CREATE TABLE IF NOT EXISTS productos (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  eliminado TINYINT(1) NOT NULL DEFAULT 0,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  nombre VARCHAR(100) NOT NULL,
  precio DOUBLE NOT NULL,
  descripcion VARCHAR(255),
  stock INT NOT NULL DEFAULT 0,
  imagen VARCHAR(255),
  disponible TINYINT(1) NOT NULL DEFAULT 1,
```

```

    categoria_id BIGINT NOT NULL,
    FOREIGN KEY (categoria_id) REFERENCES categorias(id)
);

CREATE TABLE IF NOT EXISTS pedidos (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    eliminado TINYINT(1) NOT NULL DEFAULT 0,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    total DOUBLE NOT NULL DEFAULT 0.0,
    estado VARCHAR(30) NOT NULL,
    forma_pago VARCHAR(30) NOT NULL,
    usuario_id BIGINT NOT NULL,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
);

CREATE TABLE IF NOT EXISTS detalles_pedido (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    eliminado TINYINT(1) NOT NULL DEFAULT 0,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    cantidad INT NOT NULL,
    subtotal DOUBLE NOT NULL,
    pedido_id BIGINT NOT NULL,
    producto_id BIGINT NOT NULL,
    FOREIGN KEY (pedido_id) REFERENCES pedidos(id),
    FOREIGN KEY (producto_id) REFERENCES productos(id)
);

```

2.3 Mecanismo de Baja Lógica (Soft Delete)

Para conservar la integridad histórica de auditoría fiscal del negocio, ninguna entidad se borra físicamente de la base de datos (DELETE físico prohibido). En su lugar, se adoptó el patrón **Soft Delete**:

1. Cada tabla tiene una columna eliminado TINYINT(1) inicializada en 0 (False).
2. Al ejecutar la acción "Eliminar" en consola, el DAO ejecuta una operación de actualización:
UPDATE tabla SET eliminado = true WHERE id = ?.
3. Las consultas de listado se filtran activamente en SQL para ignorar los registros dados de baja:
SELECT * FROM tabla WHERE eliminado = false.

2.4 Mapeo de Datos SQL a Objetos Java

El pasaje de tipos relacionales de SQL a tipos orientados a objetos de Java se realiza en los DAOs a través de JDBC:

- BIGINT → java.lang.Long
- TINYINT(1) → boolean
- DATETIME → java.time.LocalDateTime utilizando rs.getTimestamp().toLocalDateTime()
- VARCHAR → java.lang.String
- VARCHAR (Roles/Estados) → Conversión a Enums en Java mediante Rol.valueOf(rs.getString("rol"))

2.5 Arquitectura de Excepciones Personalizadas

Para un control fino del flujo de ejecución, se creó un ecosistema de excepciones personalizadas de tipo *Checked Exception* (heredando de *Exception*). Esto obliga contractualmente a que el programador capture de manera explícita el error mediante bloques try-catch y permite separar lógicas ante fallos de usuario:

- CamposVaciosException: Lanzada cuando el servicio detecta un intento de persistir strings vacíos o nulos.
- EmailDuplicadoException: Lanzada cuando la validación lógica detecta un mail idéntico activo en la base de datos (evitando violar la restricción UNIQUE de MySQL).
- UsuarioNoEncontradoException: Lanzada al intentar editar, buscar o eliminar de manera lógica una entidad que no tiene un registro asociado activo en base de datos.

2.6 Robustez del Menú en Consola

El Main controla el ciclo de vida del menú por consola impidiendo cierres inesperados. Toda entrada capturada mediante Scanner.nextLine() es sometida a parseos defensivos y validación de expresiones. Los errores de ingreso de caracteres alfabéticos en opciones numéricas (NumberFormatException) se capturan en el menú de presentación, informando al usuario y retornando de forma segura a la pantalla del menú anterior.

3. DIFICULTADES Y SOLUCIONES

A lo largo del ciclo de desarrollo surgieron varios desafíos de integración y compatibilidad técnica. A continuación se detallan las principales dificultades y cómo el equipo logró resolverlas de manera eficiente.

3.1 Transición Metodológica de Colecciones a JDBC Real

- **Dificultad:** La consigna original del trabajo práctico estipulaba el uso de colecciones en memoria de Java. Al migrar a un entorno persistido con MySQL, la clase base abstracta original y los constructores debían modificarse, ya que en memoria los IDs debían autoincrementarse manualmente, mientras que en MySQL los IDs se delegan a la cláusula AUTO_INCREMENT del motor de base de datos.
- **Solución:** Se reestructuraron los constructores en la clase Base y sus heredadas para proveer una doble vía: un constructor sin ID para el instanciado limpio en UI (delegando el ID nulo para que MySQL lo genere automáticamente en el INSERT), y otro constructor parametrizado completo que permite al DAO reconstruir el objeto de Java con el ID que ya fue asignado físicamente en la consulta de lectura de base de datos (rs.getLong("id")).

3.2 Bug del Scanner: Excepciones de Entrada Vacía en Menús Numéricos

- **Dificultad:** Durante las pruebas del menú de creación de usuarios, el operador dejaba campos de texto vacíos de manera deliberada. Al llegar a la selección numérica del rol (Integer.parseInt(scanner.nextLine())), un Enter vacío provocaba el lanzamiento inmediato de una excepción nativa NumberFormatException ("For input string: \"\""). Dado que este error ocurría en la interfaz de usuario, interrumpía el proceso de ejecución saltándose la excepción personalizada CamposVaciosException desarrollada en la capa del servicio.
- **Solución:** Se implementó una lógica de lectura defensiva en el Main. El valor numérico se lee inicialmente como una cadena genérica String. Si la cadena está vacía, se transfiere un valor de inicialización nulo (null) a los atributos del objeto de dominio. De esta manera, el objeto se instancia completamente en el Main y es transferido al Service, donde finalmente la regla de validación de campos vacíos puede evaluar el campo nulo y arrojar prolijamente un CamposVaciosException capturado correctamente por el bloque try-catch específico del operador de la consola.

3.3 Integridad Referencial y Manejo de Stock Transaccional

- **Dificultad:** Al crear un pedido con múltiples renglones (DetallePedido), existía el riesgo de que al ingresar la cantidad de un producto que no poseía stock, el pedido quedara "a mitad de camino", registrando un pedido vacío o inconsistencias entre el total financiero calculado en Java y la base de datos real.

- **Solución:** Se implementó una validación previa en cascada dentro de PedidoService. Antes de ejecutar cualquier inserción de la cabecera del Pedido en el motor relacional, se verifica mediante un bucle en memoria el stock físico disponible en la base de datos de cada uno de los productos seleccionados empleando el DAO. Si uno de los productos no cumple con las unidades disponibles requeridas, el servicio detiene la operación antes de interactuar con la base de datos y lanza un StockInvalidoException. Esto garantiza un comportamiento atómico en el flujo de negocio del sistema.

3.4 Problemas con el DetallePedido

- **Dificultad:** Durante la creación de pedidos, se presentaba un error de *Lock wait timeout* debido a una gestión inadecuada de transacciones. El método de guardado de PedidoDAOImpl abría una conexión, mientras que DetallePedidoDAOImpl intentaba abrir una segunda conexión independiente, generando un *deadlock* al competir por los mismos recursos bloqueados.
- **Solución:** Se implementó una solución basada en el principio de atomicidad transaccional: se modificó la firma de DetallePedidoDAO.guardar() para aceptar un objeto Connection. De esta forma, el método reutiliza la conexión principal ya iniciada en PedidoDAOImpl, asegurando que todas las operaciones de inserción se ejecuten en un mismo hilo transaccional y eliminando el bloqueo.

4. DEMOSTRACIÓN DEL SISTEMA (CAPTURAS DE PANTALLA)

Nota del Desarrollador: En esta sección se deben insertar las capturas de pantalla tomadas durante el funcionamiento de la interfaz de consola, abarcando:

1. *Menú principal de opciones y navegación.*

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione una opción:
```

```
--- GESTIÓN DE CATEGORÍAS ---
1. Listar Categorías
2. Crear Categoría
3. Editar Categoría
4. Eliminar Categoría
0. Volver al menú principal
```

2. Demostración del Listado de Usuarios activos (HU-USR-01).

```
--- GESTIÓN DE USUARIOS ---
1. Listar Usuarios
2. Crear Usuario
3. Editar Usuario
4. Eliminar Usuario
0. Volver al menú principal
Seleccione: 1

Listado de Usuarios:
Usuario [ID: 2 | L, M | Email: ml@email.com | Celular: 2612580511 | Rol: ADMIN]
Usuario [ID: 3 | Mo, Ni | Email: niM@email.com | Celular: 6655423111 | Rol: USUARIO]
Usuario [ID: 4 | Agu, Lau | Email: Lau@email.com | Celular: 2617777888 | Rol: ADMIN]
```

3. Ejemplo de validación ante la creación de un usuario con campos vacíos (CamposVaciosException).

```
=== Crear Nuevo Usuario ===
Nombre:
Apellido:
Email:
Celular:
Contraseña:
Seleccione Rol (1. ADMIN / 2. USUARIO):
✗ Error de validación: Ninguno de los campos del formulario puede quedar vacío.
```

4. Mensaje informativo ante un correo duplicado de usuario (EmailDuplicadoException).

```
=== Crear Nuevo Usuario ===
Nombre: Mat
Apellido: Limina
Email: ml@email.com
Celular: 3331234111
Contraseña: 5456
Seleccione Rol (1. ADMIN / 2. USUARIO): 2
✗ El correo electrónico 'ml@email.com' ya está asociado a otra cuenta.
```

5. Confirmación en consola de una Baja Lógica (Soft Delete) y posterior verificación de que

no se muestra en los listados del sistema.

```
--- GESTIÓN DE USUARIOS ---
```

```
1. Listar Usuarios
2. Crear Usuario
3. Editar Usuario
4. Eliminar Usuario
0. Volver al menú principal
Seleccione: 4
```

```
Ingrese el ID del usuario a eliminar (Baja Lógica): 2
¿Está seguro de que desea eliminar este usuario? (S/N): s
¡Usuario eliminado lógicamente!
```

```
--- GESTIÓN DE USUARIOS ---
```

```
1. Listar Usuarios
2. Crear Usuario
3. Editar Usuario
4. Eliminar Usuario
0. Volver al menú principal
Seleccione: 1
```

```
Listado de Usuarios:
```

```
Usuario [ID: 3 | Mo, Ni | Email: niM@email.com | Celular: 6655423111 | Rol: USUARIO]
Usuario [ID: 4 | Agu, Lau | Email: Lau@email.com | Celular: 2617777888 | Rol: ADMIN]
```

5. BIBLIOGRAFÍA Y WEBGRAFÍA

1. Oracle Corporation. (2026). *Java Platform, Standard Edition API Specification, Release 21*. Recuperado de: <https://docs.oracle.com/en/java/javase/21/>
2. MySQL. (2026). *MySQL 8.0 Reference Manual*. Oracle. Recuperado de: <https://dev.mysql.com/doc/refman/8.0/en/>
3. HikariCP Connection Pool Project. (s.f.). *HikariCP: Light, fast, trustworthy connection pool*. GitHub Repository. Recuperado de: <https://github.com/brettwooldridge/HikariCP>
4. Bloch, J. (2018). *Effective Java* (3rd ed.). Boston, MA: Addison-Wesley Professional.
5. Freeman, E., & Robson, E. (2020). *Head First Design Patterns* (2nd ed.). Sebastopol, CA: O'Reilly Media.